

Capacitated Confluent Flows: Complexity and Algorithms

Daniel Dressler¹ and Martin Strehler²

¹ TU Berlin, Inst. f. Mathematik, MA 5-2, Str. des 17. Juni 136,
10623 Berlin, Germany

`dressler@math.tu-berlin.de`

² BTU Cottbus, Mathematisches Institut, Postfach 10 13 44,
03013 Cottbus, Germany

`strehler@math.tu-cottbus.de`

Abstract. A flow on a directed network is said to be *confluent* if the flow uses at most one outgoing arc at each node. Confluent flows arise naturally from destination-based routing. We study the Maximum Confluent Flow Problem (MAXCONF) with a single commodity but multiple sources and sinks. Unlike previous results, we consider heterogeneous arc capacities. The supplies and demands of the sources and sinks can also be bounded. We give a pseudo-polynomial time algorithm and an FPTAS for graphs with constant treewidth. Somewhat surprisingly, MAXCONF is NP-hard even on trees, so these algorithms are, in a sense, best possible. We also show that it is NP-complete to approximate MAXCONF better than $3/2$ on general graphs.

Keywords: approximation algorithm, confluent flow, network flow, routing, treewidth.

1 Introduction

Flows have been a major planning tool ever since Ford and Fulkerson [1] studied them. The basic Maximum (*Splittable*) Flow Problem is computationally easy and well understood [2]. However, many applications call for flows with restricted structures and optimizing these often leads to NP-hard problems. One well-known example are *unsplittable* flows [3]. Here, the flow from each source must travel along a single path to the sink, while normally a flow could split into many paths.

A *confluent* flow takes this one step further: At every vertex, all flow traveling through the vertex must leave along one arc. This leads to a forest of flow-carrying arcs pointing towards the sinks. In particular, every confluent flow is unsplittable, but the reverse does not hold in general.

Confluent flows can occur in many contexts, with the prime example being destination-based routing: Whenever a node decides where to route a packet based solely on the destination of the packet, all packets to the same destination converge in a tree-like pattern. This is a common property of many routing schemes in telecommunications. While this simplifies the packet forwarding,

managing the protocols that determine the best next hop in a changing network is the hard part [4]. Destination-based routing is also used in railway logistics [5] and even emergency exit signs can be interpreted that way. Furthermore, certain virtual private network design problems always admit a flow-carrying tree as the optimum solution [6].

In this article, we consider the Maximum Confluent Flow Problem (denoted MAXCONF), where the goal is to send the maximum amount of flow from the sources to the sinks in a confluent way. The notable difference to previous works is that the flow has to obey heterogeneous arc capacities. Consequently, not all supplies may be routable. Furthermore, the sources and sinks may have upper bounds on their supplies or demands.

1.1 Related Work

Even though confluent flows appear in many applications, there is little mathematical literature discussing the basic underlying problems. Chen et al. [7,8] introduce confluent flows and study the objective of minimizing the maximum node congestion (i.e., the maximum flow into a node, which always occurs at a sink). They derive a powerful inapproximability result and also derive an almost matching approximation algorithm. Furthermore, the authors consider a maximum flow variant, where each vertex must send exactly 0 or its supply, and give a 13.29-approximation. All of this is effectively for uniform capacities and the authors list heterogeneous capacities among their open problems.

Similarly to the node congestion problems, network design problems usually adjust the capacities of the edges to route *all* demands and minimize the costs of the edges. Thus, while there are many algorithmic results for these, their objective is quite different from the MAXCONF.

As we use dynamic programming in our approach, let us refer to Bodlaender's compact introduction to dynamic programming on graphs with bounded treewidth [9].

1.2 Our Results

We present a pseudo-polynomial time algorithm for MAXCONF for graphs with bounded treewidth. We also derive an FPTAS in that case, i.e., a $(1 - \varepsilon)$ -approximation algorithm with a running time polynomial in the input and $1/\varepsilon$. We match this with a complexity result, implying that the problem is weakly NP-hard on graphs with bounded treewidth. We show that MAXCONF on general graphs cannot be approximated within a factor of $3/2 - \varepsilon$ unless $\mathcal{P} = \mathcal{NP}$.

1.3 Overview of the Paper

Section 2 fixes the notation. Section 3 summarizes the basic properties and complexity results. Section 4 formally defines tree decompositions, which allows us to explain the main idea behind the algorithm in Section 5. Then the core algorithm is developed in Section 6, with Section 7 showing the main results. Note that the more technical proofs are omitted due to space constraints.

2 Preliminaries

Let $G = (V, A)$ denote a directed graph without multiple arcs or loops. We use $A(G)$ to denote the arc set of a graph. For $V' \subseteq V$, let $G[V']$ denote the subgraph of G induced by V' and let $A[V'] := A(G[V'])$. For $v \in V$ let $\delta^+(v) := \{(u, w) \in A : u = v\}$ and $\delta^-(v) := \{(w, u) \in A : u = v\}$ be the outgoing and incoming arcs of v , respectively. Let $\mathcal{F}(V')$ be the family of sets $F \subseteq A[V']$ such that $|F \cap \delta^+(v)| \leq 1$ for all $v \in V'$.

Definition 1 (Instance). *An instance of a flow problem consists of a directed graph $G = (V, A)$, a capacity function $u : A \rightarrow \mathbb{N}_0$, a set of sinks $T \subseteq V$ with sink capacities $c : T \rightarrow \mathbb{N}_0 \cup \{\infty\}$, and upper bounds on the supplies $d : V \rightarrow \mathbb{N}_0 \cup \{\infty\}$. The vertices $S := \{v \in V : d(v) > 0\}$ are called sources and we assume $S \cap T = \emptyset$.*

Definition 2 (Flow). *Given an instance, a flow is a function $x : A \rightarrow \mathbb{R}$ with $0 \leq x(a) \leq u(a)$ for all $a \in A$. A flow is integral if it has only integral values. The inflow and outflow of a flow x at a given node v are $\text{inflow}_x(v) := \sum_{a \in \delta^-(v)} x(a)$ and $\text{outflow}_x(v) := \sum_{a \in \delta^+(v)} x(a)$. Let $\text{bal}_x(v) := \text{outflow}_x(v) - \text{inflow}_x(v)$ be the balance of a node. A flow x is an S - T -flow if it satisfies the flow conservation constraint $0 \leq \text{bal}(v) \leq d(v)$ for all $v \in V \setminus T$ and $-\text{bal}_x(t) \leq c(t)$ for all $t \in T$. The value of an S - T -flow is $\text{val}(x) := -\sum_{t \in T} \text{bal}(t)$ (as we assume $d(t) = 0 \ \forall t \in T$).*

Definition 3 (Confluent flow). *Given an instance and a flow x , let $F_x := \{(v, w) \in A : x(v, w) > 0\}$ be the set of flow-carrying arcs. A flow x is called nearly confluent if $F_x \in \mathcal{F}(V)$ and $F_x \cap \delta^+(t) = \emptyset$ for all $t \in T$. A nearly confluent flow x is called confluent if F_x contains no directed cycle.*

Now we can formally define MAXCONF.

Problem 1 (MaxConf). INPUT: An instance G, u, d, S, T and c .
OUTPUT: The maximum flow value over all confluent S - T -flows.

3 Understanding the Problem

First consider a nearly confluent flow x and the set of used arcs F_x . This set may contain directed cycles. However, there is no undirected cycle in F_x that is not a directed cycle as well, because it would contain a vertex with two outgoing arcs, which is not allowed in F_x . The flow caught in a directed cycle can never contribute to the flow value because sinks have no outgoing arcs and cannot lie on such a cycle. So the maximum value of a nearly confluent S - T -flow also solves the MAXCONF problem.

If x is indeed a confluent flow, then it cannot contain directed cycles, either. Thus, F_x is always a forest pointing towards the sinks. In any case, the flow value of a (nearly) confluent flow simplifies to $\sum_{t \in T} \text{inflow}_x(t)$, because no flow may leave the sink.

It is a helpful observation that for integral input data, there is always an integral maximum confluent S - T -flow x . If x is not integral, one can consider the basic maximum (splittable) S - T -flow on F_x instead, which can be chosen integral. It is confluent and has equal value. The same applies to nearly confluent flows.

We further assume that G is a simple directed graph with no loops or parallel arcs. This does not impose any real restriction on the graph: Loops can never contribute to a solution and all but the one parallel arc with the highest capacity are redundant. Finally, one can also model undirected graphs by using two anti-parallel arcs, because the resulting 2-cycles do not affect the solution, either.

Let us now fix the decision version of MAXCONF to discuss its hardness: “Given an instance G , u , d , S , T and c , is there a confluent S - T -flow with value at least f ?”.

Chen et al. [8] study a variant of this problem where the objective is to minimize the node congestion, i.e. the maximum flow arriving at one of the sinks. One can use MAXCONF to model the decision version of their problem using sink capacities, but their inapproximability within a factor of $\frac{1}{2} \lg |T| - \varepsilon$ does not carry over. The crucial problem is that in the congestion case all demands are satisfied proportionally. As long as this proportion is 1, i.e., all flow must arrive at the sinks, the MAXCONF problem behaves the same. For lower values, MAXCONF does not enforce any proportion at all. Furthermore, their demand maximization variant, an “all-or-nothing flow”, cannot be approximated within a factor of $2 - \varepsilon$. But for MAXCONF the supplies at the sources may be satisfied partially and thus this result does not carry over, either. However, we still obtain the following complexity results. The latter two also carry over to undirected graphs.

- Theorem 1.** 1. *Approximating MAXCONF on general graphs within a factor of $3/2 - \varepsilon$ is NP-hard.*
2. *MAXCONF is NP-hard if G is a directed tree and all source and sink capacities are infinite.*
3. *MAXCONF with a single sink is NP-hard on planar graphs with treewidth 2 (cf. Section 4 for the definition of treewidth).*

Proof. 1. One can model the k -DISJOINT PATHS Problem as a confluent flow: Given a directed graph G with k terminal pairs $(s_1, t_1), \dots, (s_k, t_k)$, are there k node-disjoint paths that connect the corresponding terminals? Even for $k = 2$, this problem is strongly NP-complete [10]. It can be modelled as a MAXCONF on the same graph with arc capacities $u \equiv 2$. Each source s_i , $i \in \{1, 2\}$, has supply $d(s_i) := i$ and each sink t_i has capacity $c(t_i) := i$. The maximum flow value is 3 if and only if the two disjoint paths exist, because whenever two paths meet, one sink cannot be used at all. Otherwise, the optimum value is at most 2.

2. We reduce the weakly NP-complete PARTITION problem [11] to the decision version of MAXCONF. An instance to PARTITION consists of n natural numbers $d_1, \dots, d_n$ with $D := \sum_{i=1}^n d_i$ and the question is, whether there is a subset $P \subseteq \{1, \dots, n\}$ such that $\sum_{i \in P} d_i = D/2$. The input graph for MAXCONF is described in Figure 1, with infinite supplies at all s_i and 0

otherwise. The sinks $T = \{t_1, \dots, t_n, t^*\}$ also have infinite capacities. The arc capacities are given in the figure.

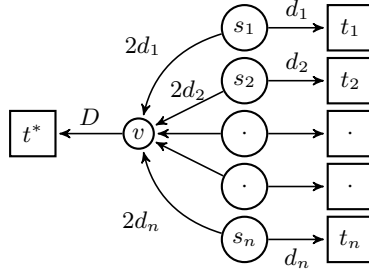


Fig. 1. The graph to model the PARTITION instance

We claim that the solution to MAXCONF is at least $f := \frac{3}{2}D$ if and only if the PARTITION instance is a yes-instance. We can identify $P \subseteq \{1, \dots, n\}$ with the sources sending to t^* in an optimum solution. Let $z := \sum_{i \in P} d_i$. Then the value of the flow is $\min\{2z, D\} + (D - z)$, which attains its unique maximum $\frac{3}{2}D$ at $z = D/2$. Thus, the optimum value is $\frac{3}{2}D$ if and only if a partition exists.

3. Note that in the last proof we can identify all of T with t^* . This yields a planar graph that has treewidth 2 with only one sink. $\square$

4 Tree Decompositions

This discouraging result leads to an examination of special graph classes. Our dynamic program uses the special structure of graphs with bounded treewidth to solve MAXCONF. Using the underlying tree structure of a graph one can solve the MAXCONF problem in a bottom-up strategy from the “leaves” to the “root”. For defining treewidth, we follow Bodlaender [9]. In our case, the treewidth of a directed graph only depends on the underlying undirected graph with edge set $E := \{\{v, w\} : (v, w) \in A\}$, so the following definitions deal with undirected graphs. Johnson et al. [12] also give a generalization of treewidth to directed graphs, but it leads to the same results in the case that all reverse arcs are assumed to be in G as well.

Definition 4 (Tree decomposition). Let $G = (V, E)$ be an undirected graph. A tree decomposition of G is a tree $(I, \mathcal{T})$ on some new node set I with edges $\mathcal{T}$, and a family of sets $B_i \subseteq V$ for $i \in I$. The following three properties must hold:

1. $\bigcup_{i \in I} B_i = V$;
2. for every edge $\{u, v\} \in E$, there is an $i \in I$ with $\{u, v\} \subseteq B_i$;
3. for all $i, j, k \in I$, if j lies on the unique path between i and k in $(I, \mathcal{T})$, then $B_i \cap B_k \subseteq B_j$.

Each B_i is called a bag. The width of a tree decomposition is $\max_{i \in I} |B_i| - 1$. The treewidth of G is the minimum k such that G has a tree decomposition of width k .

The following additional requirements reduce the number of cases that have to be considered by the algorithm. Similar conditions are often used.

Definition 5 (Nice tree decomposition). Let $(I, \mathcal{T})$ with $\{B_i : i \in I\}$ be a tree decomposition of $G = (V, E)$ and denote a vertex $t \in V$. Then this tree decomposition is called nice if $(I, \mathcal{T})$ is a binary tree for some root r , satisfying the following additional properties:

4. If a node $i \in I$ has two children $j, k \in I$, then $B_i = B_j = B_k$. In this case, i is called a JOIN node.
5. If a node $i \in I$ has one child j , then one of the following must hold:
 - (a) $|B_i| = |B_j| - 1$ and $B_i \subseteq B_j$. Then i is called a FORGET node.
 - (b) $|B_i| = |B_j| + 1$ and $B_j \subseteq B_i$. Then i is called an INTRODUCE node.
6. $|B_i| = 1$ holds for all leaves i of $(I, \mathcal{T})$. Such an i is called a LEAF node.
7. $B_r = \{t\}$ holds.

Determining the treewidth of a graph is NP-hard. However, if the treewidth is bounded by a constant k , many related problems can be solved in polynomial or even linear time. In particular, one can check in linear time whether a graph has treewidth at most k . One can also construct a tree decomposition with width k in linear time, if it exists. However, in all cases the hidden factors are large and grow quickly with k . See [9] for an overview of fast algorithms.

Lemma 1. Let $G = (V, E)$ be a graph of treewidth k and $t \in V$ an arbitrary node. Then G has a nice tree decomposition $(I, \mathcal{T})$ of the same width with root r such that $B_r = \{t\}$ and it can be constructed in linear time, assuming that k is constant. Furthermore, the number of nodes in I is polynomially bounded in the size of G .

Proof. We assume a tree decomposition of polynomial size is given and make it “nice”. Showing that the result remains a tree decomposition of G mostly revolves around condition 3 from above. An equivalent condition requires that the nodes containing a given vertex v in their bags must form a connected component of $(I, \mathcal{T})$. This condition can be easily checked because we will only modify the tree locally.

First, we choose an arbitrary node r' with $t \in B_{r'}$ and orient the tree towards r' . If $|B_{r'}| > 1$, we introduce a new node r above r' that becomes the real root with $B_r = \{t\}$. If $|B_{r'}| = 1$, no such additional step is needed. Next we want to obtain a binary tree. We replace any node $i \in I$ with $p > 2$ children by a binary tree $(I', \mathcal{T}')$ with p leaves and all nodes $i' \in I'$ are associated with the vertex set B_i . The p children of i are then connected to the p children of the new binary tree $(I', \mathcal{T}')$. Note that the replacement binary tree can be chosen such that $|I'| \leq 2p - 1$. Thus, all such replacements can only double the size of the entire tree.

It is easy to see that leaves with empty bags may be deleted. Now consider a leaf i with $|B_i| > 1$. We simply add a new child node to it with a bag containing only one vertex out of B_i .

Notice that a single edge $\{i, k\}$ in the tree $(I, \mathcal{T})$ can be subdivided easily as long as the new node j between i and k has a bag satisfying $B_i \cap B_k \subseteq B_j \subseteq B_i \cup B_k$. So if i is a father of k that does not satisfy the bag size restrictions of a nice tree decomposition, this allows us to first introduce a node j with $B_j = B_i \cap B_k$ inbetween, and then “forget” all vertices in $B_i \setminus B_j$ one by one in further subdivisions of the edge $\{i, j\}$. Proceeding from j downwards, we can then “introduce” all vertices in $B_k \setminus B_j$ in subdivisions of $\{j, k\}$. For a JOIN node i it might actually be necessary to introduce one more subdivision to first obtain a child with the exact same bag B_i . The number of steps needed for each sequence of subdivisions is clearly bounded in $|V|$, giving a tree decomposition still with a polynomial number of nodes. Since we never use a bag larger than $\max\{|B_i|, |B_k|\}$, the treewidth does not increase. It only remains to contract edges where both ends have outdegree one and identical associated bags. $\square$

5 The Dynamic Programming Table

We now explain the overall structure of our dynamic program for solving the MAXCONF problem. First, one preprocesses the instance of MAXCONF to obtain one with finite supplies d and a single uncapacitated sink t . Then one determines a nice tree decomposition with t in the root bag. Such a tree decomposition has the useful property that each bag B_i of a node i separates the sink from the subgraph of G induced by all bags B_j of nodes j in the subtree of i (the so-called *bagend*). One can then compute suitable representations of all possible flows on each bagend, starting at the LEAFs and working towards the root. Having done so, it just remains to read off the maximum flow value at the root node.

Definition 6 (Bagend). *Given a nice tree decomposition $(I, \mathcal{T})$ and $i \in I$, the bagend G_i is the subgraph of G induced by the union of B_i and all B_j for j in the subtree rooted at i .*

The bags and bagends of a nice tree decomposition change towards the sink in an orderly fashion, so this really suggests a dynamic programming approach. However, one has to carefully choose representatives for the flows on each bagend G_i , while keeping enough information to develop the values for the next bag from these.

The main trick to make such a representation work is the following: The moment a vertex is first considered, one fixes the amount of flow that this vertex sends out. This is what we will call the *estimated outflow* $d^+(v)$. Of course, in a dynamic program we can try all possible integer values for each vertex (up to a reasonable bound U) and later on delete those combinations that turn out to be infeasible.

Flow conservation requires that a vertex cannot send more flow than it receives (including its own supply), but when a vertex is first examined, it is not clear how much flow will eventually enter this vertex. Thus, flow conservation cannot

be established immediately. Instead, we track how much more flow must reach a vertex in its *balance deficit*, which we denote by $d^-(v)$. The supply $d(v)$ does not come into play until the very last moment when a vertex leaves the currently considered bag.

The main advantage of fixing the estimated outflow is that we can locally manipulate the set of flow-carrying arcs. For example, if a vertex v uses no outgoing arc so far and we want to send the flow along (v, w) , we can simply do so without triggering a chain reaction of increasing flow along the path that these flow units take. Indeed, such a path could lead into “uncharted territory” of the graph, which would make tracking the changes even harder. Instead, we simply update the balance deficit $d^-(w)$ of the receiving vertex.

We also need to keep track of some structural information for each B_i , the most obvious being the set of used arcs $F \subseteq A[B_i]$, which coupled with d^+ imply the actual flow on $A[B_i]$. On a very abstract level, one could misleadingly think of the following idea: a tree-like flow on a tree-like graph structure pointing towards the sink should only flow in the same direction as the underlying tree decomposition. However, this turns out to be wrong, and the optimum flow may send flow back and forth between B_i , the rest of the bagend $G_i \setminus B_i$ and the part with the sink $G \setminus G_i$. The implication is that we have to remember whether a vertex has an outgoing arc at all, because this might not be clear from F , if a flow-carrying outgoing arc points to an already “forgotten” vertex (an already processed vertex in $G \setminus B_i$). For this, each vertex $v \in B_i$ is assigned one of two states: $z(v) \in \{\text{FREE}, \text{NONFREE}\}$. The state NONFREE denotes a vertex that already has an outgoing arc and satisfies the balance condition as if it had supply equal to its balance deficit $d^-(v)$. Vertices in the other state FREE resemble sinks, which is why they must have no outgoing arcs yet. These FREE vertices act as temporary storage. The estimated outflow $d^+(v)$ for a FREE vertex v means how much flow it *could* possibly send.

All of this culminates in the following table for each bag B_i :

Definition 7 (Table). Let $G = (V, A)$ be a graph with arc capacities $u : A \rightarrow \mathbb{N}_0$, supplies $d : V \rightarrow \mathbb{N}_0$ and an uncapacitated sink t . Assume that $\delta^+(t) = \emptyset$ and that $U \in \mathbb{N}_0$ is an upper bound on MAXCONF for this instance. Let $(I, \mathcal{T})$ and $(B_i)_{i \in I}$ be a nice tree decomposition of G .

For $i \in I$, let $\text{TABLE}(i)$ be defined as the set containing exactly the elements (F, z, d^-, d^+) with $F \in \mathcal{F}(B_i)$, $z \in \{\text{FREE}, \text{NONFREE}\}^{B_i}$, $d^- \in \{0, 1, \dots, U\}^{B_i}$, $d^+ \in \{0, 1, \dots, U\}^{B_i}$ for which there is an integral nearly confluent flow x in G_i with the following properties:

1. $a \in F \iff x(a) > 0 \quad \forall a \in A[B_i]$
2. $d^+(v) = \text{inflow}_x(v) + d^-(v) \quad \forall v \in B_i$
3. $\text{outflow}_x(v) = 0 \iff z(v) = \text{FREE} \quad \forall v \in B_i$
4. $d^+(v) = \text{outflow}_x(v) \quad \forall v \in B_i : z(v) = \text{NONFREE}$
5. $0 \leq \text{bal}(v) \leq d(v) \quad \forall v \in G_i \setminus B_i$.

For brevity, we use $\text{SET}(i) := \mathcal{F}(B_i) \times \{\text{FREE}, \text{NONFREE}\}^{B_i} \times \{0, 1, \dots, U\}^{B_i} \times \{0, 1, \dots, U\}^{B_i}$ to denote the domain of $\text{TABLE}(i)$.

Please note that given a nearly confluent flow x on the bagend G_i and some (F, z, d^-, d^+) , it is easy to check whether they match. However, x does not necessarily determine a unique entry in $TABLE(i)$. For a FREE vertex v , the flow only determines the difference $d^+(v) - d^-(v) = \text{inflow}_x(v)$ by condition 2, not the absolute values. For a NONFREE vertex, condition 4 eliminates any ambiguity.

6 Computing the Tables

For computing the entries of the $TABLE(i)$ out of the tables of the children of i , algorithms that address the four classes of nodes LEAF, INTRODUCE, JOIN, and FORGET are needed. One has to ensure, that the flow values are calculated correctly, and to proof, that all entries can be derived in this way. Due to space limitations we omit the very technical lemmata and lengthy proofs here. They can be found in the full version of this paper. Exemplarily, we present the easiest case of calculating $TABLE(i)$ of a LEAF node, which only contains one vertex and cannot contain any arcs at all.

Lemma 2. *Let $i \in I$ be a LEAF node.*

Then $TABLE(i) = \{(\emptyset, (\text{FREE}), (f), (f)) : 0 \leq f \leq U\}$.

Proof. If i is a LEAF in $\mathcal{T}$, then $B_i = \{v\}$ by definition. Since G_i is just a single vertex, $A(G_i)$ is empty. We show the equality by showing both inclusions.

“ $\supseteq$ ”: Let $0 \leq f \leq U$. To show that $(\emptyset, (\text{FREE}), (f), (f))$ is in $TABLE(i)$ we have to give some nearly confluent flow x and then check the conditions for $TABLE(i)$ given in Definition 7. Since $A(G_i) = \emptyset$, this flow x is a function on an empty set and thus there is no choice to be made. It is also trivially a nearly confluent flow. Note that $\emptyset$ is the set of flow-carrying arcs of $A(G_i)$, as required by condition 1. Also, $d^+(v) = f = d^-(v)$ and $\text{inflow}_x(v) = 0$, so condition 2 is satisfied. Since $\text{outflow}_x(v) = 0$, we set $z(v) = \text{FREE}$ in concordance with condition 3. Finally, there are no NONFREE vertices in this bag and no vertices in the empty set $G_i \setminus B_i$, either, so conditions 4 and 5 are trivially satisfied.

“ $\subseteq$ ”: Now let $(F, z, d^-, d^+) \in TABLE(i)$ and let x be a corresponding nearly confluent flow on G_i . (Again, x is defined on the empty set $A(G_i)$.) Since $F \subseteq A(G_i) = \emptyset$, we know $F = \emptyset$. Also, $\text{inflow}_x(v) = 0$ which, combined with condition 2, implies $d^+(v) = d^-(v) =: f \in \{0, 1, \dots, U\}$. Finally, condition 3 implies $z(v) = \text{FREE}$. Thus, every entry must have the form $(\emptyset, (\text{FREE}), (f), (f))$ as claimed and this concludes the proof. $\square$

The remaining types of nodes require a much more complex handling. We just sketch the basic ideas here. INTRODUCE nodes are algorithmically handled in two steps. First, the new vertex v is added to the bag as a FREE vertex without any flow entering or leaving it. Afterwards we can designate some of the arcs incident to v to carry flow to or from v , say along (w, w') . This must *change* the state of w to NONFREE and affects the balance of the receiving vertex w' . Of course, one needs to ensure that the arc capacity is not exceeded and that the receiving vertex actually requires that much additional incoming flow, i.e. $d^+(w) \leq \min\{u(w, w'), d^-(w')\}$ must hold.

For a JOIN node i with children j and k , we “sum” selected entries of $TABLE(j)$ and $TABLE(k)$ in such a way, that one of the children carries no flow at all on the bag $B_i = B_j = B_k$. Note, that the bagends G_j and G_k are disjoint elsewhere. Thus, we do not have to worry about exceeding capacities there, or changing the already fixed value d^+ .

It remains to describe the process for FORGET nodes. In this case, the vertex v that leaves the bag suddenly needs to satisfy any remaining imbalance $d^-(v)$ with its own supply $d(v)$, if it uses an outgoing arc at all. Otherwise, it should not try to send any flow.

Armed with algorithms for the four node classes, one can then formulate a dynamic program that computes all tables, represented as arrays containing $|SET(i)| \in \mathcal{O}(2^{(k+1)^2} 2^{k+1} U^{2(k+1)}) = \mathcal{O}(U^{2k+2})$ bits. Thus, we obtain the following theorem:

Theorem 2. *There is an algorithm that given a nice tree decomposition (I, T) computes all tables correctly and runs in time polynomial in $|I|$ and U , assuming that the width k of (I, T) is constant.*

7 Main Results

We can now apply Theorem 2 to solve the MAXCONF problem as follows.

Theorem 3. *MAXCONF on graphs with treewidth bounded by some constant k can be solved in pseudo-polynomial time.*

Proof. Consider an instance $G = (V, A)$, u , d , S , T and c . It is assumed that the treewidth of G is at most k . W.l.o.g. we can assume that no vertex has larger supply than it could send to a sink by itself. This can be assured in polynomial time by solving the BOTTLENECK SHORTEST PATH problem, e.g., with a modified Dijkstra’s algorithm (cf. [13]). In particular, these values are finite.

First we preprocess the instance to obtain a graph G' with a single supersink t^* : Remove all arcs leaving the sinks. Then add a supersink t^* and connect each old sink t to it by an arc (t, t^*) with capacity $\min\{c(t), \sum_{(v,t) \in A} u(v, t)\}$. The supersink has infinite sink capacity and no supply, while the old sinks T now become normal vertices, i.e. $T' = \{t^*\}$. Notice that this preprocessing does not change the solution of MAXCONF. Also, the treewidth of this new instance is at most $k + 1$ because a tree decomposition of G can be turned into a tree decomposition of G' by adding t^* to every bag.

Next we compute a nice tree decomposition of G' with width at most $k + 1$ and with polynomially many nodes, as described in Lemma 1. Finally, we run the dynamic program from Theorem 2 on the modified instance with $U := \sum_{v \in V} d(v)$ to compute $TABLE(r)$ for $B_r = \{t^*\}$. Note that the overall running time is pseudo-polynomial in the size of G .

We claim that the solution is the maximum value $f \in \{0, 1, \dots, U\}$ such that $(\emptyset, (\text{FREE}), (0), (f)) \in TABLE(r)$. This value can be extracted from the table easily. To see that it is indeed the solution, first recall that the maximum

value of a nearly confluent S - T' -flow is the same as the maximum value of a confluent S - T' -flow. Note that $G_r = G$ and that every nearly confluent S - T' -flow x induces an entry in $TABLE(r)$ of the form $(\emptyset, (\text{FREE}), (0), (\text{bal}_x(t^*)))$. On the other hand, all table entries of this kind correspond to a nearly confluent flow x which satisfies the flow conservation constraint in every vertex except t^* . Since $d^-(t^*) = 0$, we have $d^+(t^*) = \text{inflow}_x(t^*)$ and $\text{outflow}_x(t^*) = 0$ because $z(t^*) = \text{FREE}$, so the value of the flow must be equal to $d^+(t^*)$. $\square$

Even better, this yields an FPTAS for MAXCONF.

Theorem 4. MAXCONF on graphs with $|V|$ nodes and treewidth bounded by some constant k can be approximated to a factor of $(1 - \varepsilon)$ for any $\varepsilon > 0$ in time polynomial in $|V|$ and $1/\varepsilon$.

Proof. Again the input is $G = (V, A)$, u , d , T and c and we assume that the supplies d are all individually routable. We first have to bound the value f^* of an optimal solution. With $d_{\max} := \max_{v \in V} d(v)$, we obtain $d_{\max} \leq f^* \leq \sum_{v \in V} d(v) \leq |V|d_{\max} =: U$.

We will use a simple rounding scheme to obtain the approximation. Let $\varepsilon' := \varepsilon/|V|^2$ and round down all numerical values to multiples of $\varepsilon'U$. Call these new values $\tilde{u}$, $\tilde{d}$ and $\tilde{c}$. For example, $\tilde{u}(a) := \left\lfloor \frac{u(a)}{\varepsilon'U} \right\rfloor \varepsilon'U$. If we further scale the instance by $(\varepsilon'U)^{-1}$, we obtain integral values between 0 and $\lceil 1/\varepsilon' \rceil$. We run the algorithm to solve this scaled down integral instance. Theorem 3 shows that the running time is polynomial in the size of the graph and $\tilde{U} := (\varepsilon'U)^{-1} \sum_{v \in V} \tilde{d}(v) \in \mathcal{O}(|V|/\varepsilon') = \mathcal{O}(|V|^3/\varepsilon)$. Thus, this is a polynomial time algorithm overall.

Let f denote the optimum value of the rounded instance scaled back to the original range by a factor of $\varepsilon'U$. (The optimum value scales with the parameters.) We need to show $f \geq (1 - \varepsilon)f^*$ to conclude this proof.

Consider a confluent S - T -flow x with value f^* for the original instance. Standard flow theory tells us that we can decompose it into at most $|F_x| \leq |V|$ paths $P \in \mathcal{P}$ carrying flow from the sources to the sinks. (Cycles cannot occur in F_x .) We denote the flow on each path with $x(P)$, and $\sum_{P \in \mathcal{P}} x(P) = f^*$ must hold. Rounding the flow on each path to $\tilde{x}(P) := \left\lfloor \frac{x(P)}{\varepsilon'U} \right\rfloor \varepsilon'U$ yields a path decomposition of a new flow $\tilde{x}$ where all flow values are multiples of $\varepsilon'U$. Rounding down individual paths results in a larger loss due to rounding than for d , u or c , so $\tilde{x}$ obeys all rounded capacities. Thus, $\tilde{x}$ is a confluent S - T -flow that is feasible in the rounded instance and has value $\sum_{P \in \mathcal{P}} \tilde{x}(P) \geq \varepsilon'U \sum_{P \in \mathcal{P}} (\frac{x(P)}{\varepsilon'U} - 1) = f^* - |V|\varepsilon'U$. Since f is the optimum value of a feasible flow in the rounded instance, it follows that $f \geq \text{val}(\tilde{x}) \geq f^* - |V|\varepsilon'U$.

Recalling the bounds on f^* , we obtain $f \geq f^* - |V|\varepsilon'U = f^* - |V|^2\varepsilon'd_{\max} = f^* - \varepsilon d_{\max} \geq f^* - \varepsilon f^* = (1 - \varepsilon)f^*$ as claimed. $\square$

As usual, backtracking in the dynamic program provides the actual flow as well for both algorithms. Note that it is also possible to modify the exact dynamic program in order to handle, e.g., lower capacities on arcs or all-or-nothing flows. However, since the accuracy of the rounded instance is limited, these conditions only carry over in an $\mathcal{O}(\varepsilon)$ -precise way to the approximation algorithm.

References

1. Ford, L.R., Fulkerson, D.R.: Maximal flow through a network. *Canadian Journal of Mathematics* 8, 399–404 (1956)
2. Ahuja, R.K., Magnanti, T.L., Orlin, J.B.: *Network Flows: Theory, Algorithms, and Applications*. Prentice-Hall, Englewood Cliffs (1993)
3. Kleinberg, J.M.: Single-source unsplittable flow. In: 37th Annual Symposium on Foundations of Computer Science, pp. 68–77 (1996)
4. Bley, A.: *Routing and Capacity Optimization for IP Networks*. PhD thesis, TU Berlin (2007)
5. Fügenschuh, A., Homfeld, H., Schuelldorf, H.: Routing cars in rail freight service. *Dagstuhl Seminar Proceedings*, vol. 09261, Dagstuhl, Germany (2009)
6. Goyal, N., Olver, N., Shepherd, F.B.: The VPN conjecture is true. In: *STOC 2008: Proceedings of the 40th annual ACM symposium on Theory of computing*, pp. 443–450. ACM, New York (2008)
7. Chen, J., Rajaraman, R., Sundaram, R.: Meet and merge: approximation algorithms for confluent flows. *J. Comput. Syst. Sci.* 72, 468–489 (2006)
8. Chen, J., Kleinberg, R.D., Lovász, L., Rajaraman, R., Sundaram, R., Vetta, A.: (Almost) tight bounds and existence theorems for single-commodity confluent flows. *J. ACM* 54, 16 (2007)
9. Bodlaender, H.: Treewidth: Algorithmic techniques and results. In: Privara, I., Ružička, P. (eds.) *MFCS 1997. LNCS*, vol. 1295, pp. 29–36. Springer, Heidelberg (1997)
10. Fortune, S., Hopcroft, J., Wyllie, J.: The directed subgraph homeomorphism problem. *Theoretical Computer Science* 10, 111–121 (1980)
11. Garey, M.R., Johnson, D.S.: *Computers and Intractability, A Guide to the Theory of NP-Completeness*. W.H. Freeman and Company, New York (1979)
12. Johnson, T., Robertson, N., Seymour, P.D., Thomas, R.: Directed tree-width. *Journal of Combinatorial Theory, Series B* 82, 138–154 (2001)
13. Kaibel, V., Peinhardt, M.A.F.: On the bottleneck shortest path problem. Technical Report 06-22, Konrad-Zuse-Zentrum f. Informationstechnik Berlin (2006)